Leibniz FH

Wirtschaftsinformatik (B. Sc.)
Modul 20.1 (Vertiefung Entwicklung)
Sommersemester 2026

# Anwendung erweiterter Softwareentwicklung

## 3. Python-Grundlagen

Dr. Peter Lohmann

Die duale Hochschule für
Wirtschaft und Informatik in Niedersachsen

Leibniz FH

# 3.1. Sprach-Design

Die duale Hochschule für
Wirtschaft und Informatik in Niedersachsen

# Geschichte

- ✓ Anfang der 1990er Jahre, Guido van Rossum
- ✓ Ziel: gut lesbare, kompakte, praktisch einsetzbare Sprache
- ✓ Einflüsse anderer Sprachen
  - ABC: gut verständliche Syntax
  - Modula-3 und *C*: pragmatische Struktur- und Implementierungsideen
  - Lisp, Haskell, *Smalltalk* u. a.: fortgeschrittene Sprachmerkmale
- ✓ Zentrales Designziel: hohe Lesbarkeit
  - bewusst klare, wenig syntaktisch überladene Konstrukte



#JoinOurNetwork

# Alles ist ein Objekt!

- ✓ Zahlen, Strings, Funktionen, Klassen: Objekte
  - Typ als Eigenschaft des Objekts
  - Laufzeitabfrage mit `type(...)`
- ✓ Einheitliche Mechanismen in der Sprache
  - Attribute, Methoden, Referenzen fast überall
- ✓ Praktische Folge
  - dieselben Werkzeuge für viele Konzepte, z. B. `type(...)`, `dir(...)`



#JoinOurNetwork

# Dynamische Typisierung

- ✓ Namen an Objekte gebunden, nicht an fest deklarierte Typen
- ✓ Praktische Folge
  - klare Benennung
  - kleine Funktionen
  - Unit-Tests



#JoinOurNetwork

# Zur Laufzeit interpretiert

- ✓ Direkte Ausführung durch den Interpreter
  - kein separater Build-Schritt
- ✓ Sehr direkter Arbeitsstil
  - REPL, Skripte, Notebooks
  - schnelles Ausprobieren, iterative Entwicklung
- ✓ Technischer Hintergrund
  - Übersetzung in Bytecode
  - Verarbeitung durch die Laufzeitumgebung



#JoinOurNetwork

# Unterschiede zu C#

- ✓ C#: statisch typisiert
- ✓ Python: dynamisch typisiert
  - – frühere Fehlererkennung in C#
  - – mehr Flexibilität in Python
- ✓ C#: kompiliert (so halb)
- ✓ Python: interpretiert
- ✓ C#: stärker formalisierte Toolchain
- ✓ Python: näher an Skript-, Analyse- und Automatisierungs-Workflows



#JoinOurNetwork

# Zen of Python - PEP 20

- ✓ https://peps.python.org/pep-0020/
- ✓ Sammlung knapper Designprinzipien
- ✓ Praktische Bedeutung
  - Orientierung für idiomatischen, gut lesbaren Python-Code
  - kein starres Gesetz, aber starke kulturelle Leitlinie

## The Zen of Python

```
Beautiful is better than ugly.
Explicit is better than implicit.
Simple is better than complex.
Complex is better than complicated.
Flat is better than nested.
Sparse is better than dense.
Readability counts.
Special cases aren't special enough to break the rules.
Although practicality beats purity.
Errors should never pass silently.
Unless explicitly silenced.
In the face of ambiguity, refuse the temptation to guess.
There should be one-- and preferably only one --obvious way to do it.
Although that way may not be obvious at first unless you're Dutch.
Now is better than never.
Although never is often better than *right* now.
If the implementation is hard to explain, it's a bad idea.
If the implementation is easy to explain, it may be a good idea.
Namespaces are one honking great idea -- let's do more of those!
```



#JoinOurNetwork

Leibniz FH

# 3.2. Grundbausteine

Die duale Hochschule für
Wirtschaft und Informatik in Niedersachsen

# Namen und Variablen

- ✓ Namen binden Objekte
  - Variablen als Bezeichner
  - nicht Speicherbox, sondern nur eine Referenz
- ✓ Identifier
  - Buchstaben, Ziffern, _ (Unterstrich)
  - kein Start mit Ziffer



#JoinOurNetwork

# Namenskonventionen - PEP 8

- ✓ https://peps.python.org/pep-0008/
- ✓ Variablen- und Funktionsnamen: `snake_case`
- ✓ Klassennamen: `CamelCase`
- ✓ Konstanten: `UPPER_CASE`
- ✓ Ziel
  - einheitlicher, gut lesbarer Code
  - weniger unnötige Stilbrüche



#JoinOurNetwork

# Built-in-Typen

- ✓ Numerisch
  - `int, float`
- ✓ Logisch
  - `bool`
- ✓ Text
  - `str`
- ✓ Speziell
  - `None` als fehlender Wert



#JoinOurNetwork

# Zusammengesetzte Typen

- ✓ Sequenzen / Arrays
  - `list, tuple, range`
- ✓ Hash-Table / Map
  - `dict`
- ✓ Mengen
  - `Set`
  - Kein sorted set in Python!
- ✓ Wichtige Frage
  - geordnet oder ungeordnet
  - Veränderlich (mutable) oder unveränderlich



#JoinOurNetwork

# Typ-Annotationen

- ✓ Zusätzliche Typinformationen im Code
  - z.B. für Parameter und Rückgabewerte
- ✓ Nutzen
  - Dokumentation
  - Tool-Unterstützung, z.B. IDE auto-complete
  - statische Analyse
- ✓ Wichtig
  - keine automatische Laufzeitprüfung durch den Interpreter



#JoinOurNetwork

# Operatoren und einfache Expressions

- ✓ Arithmetische Operatoren
  - **+, −, *, /, //, %, ****
- ✓ Zuweisung (Assignment)
  - **=**
  - **+=**, …
- ✓ Ausdrücke (Expressions)
  - liefern Werte
  - kombinieren Operanden und Operatoren



#JoinOurNetwork

# Vergleiche und Boolean-Logik

- ✓ Vergleichsoperatoren
  - `==, !=, <, <=, >, >=`
  - auch verkettete Vergleiche: `a < b < c`
    - entspricht: `a < b and b < c`
- ✓ Boolean-Operatoren
  - `and, or, not`



#JoinOurNetwork

# Typ-Konversion

- ✓ Motivation
  - flexible Verarbeitung unterschiedlicher Werte
  - Typverträglichkeit oft erst zur Laufzeit sichtbar
- ✓ Explizite Konversion
  - `int(...), float(...), str(...), list(...)`
- ✓ automatische Anpassung bei gemischten numerischen Expressions
- ✓ Mögliche Risiken
  - Informationsverlust
  - Laufzeitfehler bei ungeeigneten Werten



#JoinOurNetwork

# Duck-Typing

- ✓ Fokus auf Verhalten statt auf exakten Typ
  - wichtig ist, was ein Objekt kann
  - nicht, von welcher Klasse es stammt
- ✓ Typische Idee
  - wenn passende Methoden vorhanden, dann Objekt direkt verwendbar
- ✓ Praktische Folge
  - flexible, generische Funktionen
  - Fehler oft erst zur Laufzeit sichtbar

Leibniz FH

# 3.3. Arbeiten mit Objekten und Daten

Die duale Hochschule für
Wirtschaft und Informatik in Niedersachsen

# Container-Objekte

- ✓ Container fassen mehrere Werte zusammen
- ✓ Typische Container
  - `list, tuple, dict, set`
- ✓ Auswahl nach Zweck
  - Reihenfolge
  - Änderbarkeit
  - Schlüsselzugriff (key-based) oder Elementzugriff (index-based)



#JoinOurNetwork

# Comprehensions

- ✓ Kompakte Schreibweise für neue Container
- ✓ Typische Muster
  - Transformation
  - Filterung
- ✓ Vorteil
  - kurze, ausdrucksstarke Form
- ✓ Voraussetzung
  - Schleifenlogik klar verstehen

# Indexing und Slicing

- ✓ Indexing
  - Zugriff auf einzelne Elemente über Positionen
  - auch negative Indizes
- ✓ Slicing
  - Zugriff auf Teilbereiche
  - Form: `start:stop:step`
- ✓ Typische Anwendung
  - Listen, Tupel, Strings



#JoinOurNetwork

# Mutability

- ✓ Mutable Objekte
  - Inhalt veränderbar, z. B. list, dict, set
- ✓ Immutable Objekte
  - Inhalt nicht veränderbar, z. B. int, float, str, tuple
- ✓ Praktische Folgen
  - wichtig für Assignment
  - wichtig für Argument Passing



#JoinOurNetwork

# Identity und Membership

- ✓ Identity
  - `is, is not`
  - Vergleich der Objektidentität, nicht des Inhalts
- ✓ Membership
  - `in, not in`
  - Prüfung auf enthaltene Elemente
- ✓ Typische Fehlerquelle
  - `is` statt `==`
    - Aber manchmal korrekt, z.B.: `if var is None`



#JoinOurNetwork

Leibniz FH

# 3.4. Kontrollfluss und Blockstruktur

Die duale Hochschule für
Wirtschaft und Informatik in Niedersachsen

# Blocks und Einrückung

- ✓ Blockstruktur durch Einrückung
  - keine **{**...**}**-Blöcke wie in C#
- ✓ Einrückung als Syntax (significant whitespace)
  - konsistent und präzise notwendig
- ✓ Vorteil
  - optisch klarer Programmaufbau
- ✓ Nachteil
  - “wft?” – das gibt sich aber



#JoinOurNetwork

# Verzweigungen

- ✓ Bedingte Ausführung mit `if, elif, else, match/case`
- ✓ Bedingungen liefern `True` oder `False`
- ✓ Typische Anwendung
  - Fallunterscheidungen
  - Plausibilitätsprüfungen



#JoinOurNetwork

# Schleifen

- ✓ Wiederholte Ausführung von Code
- ✓ `while`
  - bedingungsgesteuerte Wiederholung
- ✓ `for`
  - Iteration über Container
- ✓ Häufige Hilfen
  - `range(...)`
  - `break, continue`



#JoinOurNetwork

Leibniz FH

# 3.5. Funktionen und Scope

Die duale Hochschule für
Wirtschaft und Informatik in Niedersachsen

# Funktionen

- ✓ Zentrale Abstraktion für wiederverwendbaren Code
- ✓ Definition mit `def foo_bar(buz)`
- ✓ Anonyme Kurzform mit `lambda x: 42*x`
- ✓ Ziel
  - Zerlegung in kleine, verständliche Einheiten



#JoinOurNetwork

# Rückgabewerte und Unpacking

- ✓ Ergebnisse mit `return`
- ✓ Mehrfache Rückgaben
  - technisch meist als tuple
- ✓ Unpacking
  - Zerlegung zusammengesetzter Rückgabewerte in mehrere Namen



#JoinOurNetwork

# Parameter-Arten

- ✓ Positionale Parameter
  - Zuordnung über Reihenfolge
- ✓ Keyword-Parameter
  - Zuordnung über Namen
- ✓ Default-Werte
  - optionale Argumente
- ✓ Variable Argumente
  - `*args, **kwargs`



#JoinOurNetwork

# Argument Passing

- ✓ Übergabe von Objektreferenzen
  - Bindung der Parameternamen an vorhandene Objekte
- ✓ Folge bei mutable Objekten
  - Änderungen können außerhalb sichtbar sein
- ✓ Folge bei immutable Objekten
  - eher neue Zuweisung als inhaltliche Änderung



#JoinOurNetwork

# Scope

- ✓ Lokale Namen
  - innerhalb von Funktionen
- ✓ Globale Namen
  - auf Modulebene
- ✓ Wichtige Python-Eigenschaft
  - Blockgrenzen erzeugen keinen eigenen Scope



#JoinOurNetwork

Leibniz FH

# 3.6. Module und Programme

Die duale Hochschule für
Wirtschaft und Informatik in Niedersachsen

# Module und Imports

- ✓ Module sind wiederverwendbare Python-Dateien
- ✓ `import` fürs Einbinden anderer Module/Dateien
- ✓ Vorteile
  - Strukturierung größerer Programme
  - Wiederverwendung
  - Trennung von Verantwortlichkeiten



#JoinOurNetwork

# Programme strukturieren

- ✓ Code aufteilen in Funktionen, Module, Pakete
- ✓ Klare Zuständigkeiten pro Datei
- ✓ Typische Trennung
  - Fachlogik
  - Ein- und Ausgabe
  - Startpunkt des Programms, `__main__.py`



#JoinOurNetwork